

ALP

FICHA DE EXERCÍCIOS ATIVIDADE LETIVA

Programação Orientada a Objetos

UNIDADE CURRICULAR

Ficha 3 – DOM e Eventos

FICHA

Crie uma pasta para esta ficha (sugestão de nome: F03_soundbase-dom).

Dentro dessa pasta, coloque os ficheiros disponíveis no Moodle: `index.html`, `style.css` e `soundbase-dados.js`. Crie também o ficheiro `script.js` e ligue-o ao `index.html`.

Todo o código JavaScript deve estar no ficheiro `script.js`.

Utilize `console.log()` para verificar resultados intermédios.

Esta ficha continua o cenário SoundBase. Os exercícios introduzem a manipulação do DOM com JavaScript: seleccionar elementos, alterar conteúdo e estilos, reagir a eventos e trabalhar com formulários. Leia cada exercício na totalidade antes de começar.

Estrutura do `index.html`

Seletor	Descrição
<code>#app-title</code>	Título principal da aplicação (<code><h1></code>)
<code>#track-count</code>	Parágrafo para mostrar o número de faixas
<code>#search-input</code>	Campo de texto para pesquisa
<code>#track-list</code>	Contentor <code><div></code> onde as faixas serão renderizadas
<code>#add-track-form</code>	Formulário para adicionar uma nova faixa
<code>#form-title</code> , <code>#form-artist</code> , <code>#form-duration</code> , <code>#form-genre</code>	Campos do formulário

Selecionar e Modificar Elementos

- 1) Selecione o elemento `#app-title` com `getElementById` e altere o seu `textContent` para "SoundBase".
 - De seguida, selecione o mesmo elemento com `querySelector` e confirme que obtém o mesmo resultado na consola.
- 2) Selecione o elemento `#track-count` e altere o seu `textContent` para indicar o número de faixas do catalogue.

- O resultado deve ser: "6 faixas disponíveis" (ou o número correto para o seu catálogo).
- 3) Selecione o `#search-input` e defina o seu atributo placeholder para "Pesquisar por título ou artista..." COM `setAttribute`.
- Leia de seguida esse atributo com `getAttribute` e imprima-o na consola.

Nota: `getElementById` recebe apenas o nome do id, sem `#`. `querySelector` usa seletores CSS, por isso requer `#` para ids e `.` para classes.

Renderizar Conteúdo com `innerHTML`

- 4) Crie uma função `renderTrack(track)` que recebe um objeto de faixa e retorna uma string HTML com a seguinte estrutura:

```
<div class="track-card" data-title="Blinding Lights">
  <h3>Blinding Lights</h3>
  <p>The Weeknd · Pop · 3:20</p>
  <p class="plays">8 400 plays</p>
  <button class="btn-like">♥ Like</button>
  <button class="btn-remove">Remover</button>
</div>
```

- Use template strings e as funções `formatDuration` e `getTrackInfo` do `soundbase-dados.js`.
 - A função deve retornar a string - não deve usar `innerHTML` nem `console.log` internamente.
- 5) Crie uma função `renderCatalogue(tracks)` que:
- Recebe um array de faixas como argumento.
 - Usa `map` para transformar cada faixa na sua representação HTML com `renderTrack`.
 - Junta os resultados com `join("")` e atribui o resultado ao `innerHTML` do `#track-list`.
 - No final, atualiza o `#track-count` com o número de faixas renderizadas.
 - Chame `renderCatalogue(catalogue)` para renderizar o catálogo completo.

Eventos

- 6) Adicione um event listener ao `#search-input` que reage ao evento `input`.
- A cada tecla pressionada, imprima na consola o valor atual do campo (`event.target.value`).
- 7) Adicione um event listener ao `#track-list` que reage a cliques (evento `click`) em qualquer descendente.
- Use `event.target` para identificar o elemento clicado.
 - Se o elemento clicado tiver a classe `btn-like`, imprima na consola: "Like clicado: [título da faixa]".
 - Se o elemento clicado tiver a classe `btn-remove`, imprima: "Remover clicado: [título da faixa]".
 - Para obter o título, use `event.target.closest(".track-card")` e o atributo `data-title`.

Nota: Ao adicionar um único event listener ao contendor pai (em vez de um por botão), tira-se partido da propagação de eventos - esta técnica chama-se **delegação de eventos**. É especialmente útil quando o conteúdo é gerado dinamicamente.

- 8) Implemente o comportamento do botão "Remover":

- Quando um `btn-remove` for clicado, remova a `track-card` correspondente do DOM com `remove()`.
- Após remover, atualize o `#track-count` para refletir o número de elementos que ainda estão no `#track-list`.

Formulário

- Adicione um event listener ao `#add-track-form` para o evento `submit`.
 - Use `event.preventDefault()` para impedir o recarregamento da página.
 - Imprima na consola os valores dos campos `#form-title`, `#form-artist`, `#form-duration` e `#form-genre`.
- Valide os dados do formulário antes de aceitar a submissão:
 - O `title` e o `artist` não podem estar vazios.
 - O `duration` deve ser um número inteiro positivo.
 - Se a validação falhar, imprima `"Formulário inválido."` na consola e não prossiga.
- Se a validação for bem-sucedida:
 - Crie um objeto de faixa com as propriedades `title`, `artist`, `duration`, `genre` e `plays` com o valor 0.
 - Adicione-o ao array `catalogue`.
 - Chame `renderCatalogue(catalogue)` para atualizar a lista na página.
 - Limpe o formulário com `reset()`.

Exercício Integrador

- Implemente a funcionalidade de pesquisa em tempo real:
 - Atualize o event listener do `#search-input` (exercício 6) para que, a cada evento `input`, filtre o `catalogue` com `filter`.
 - A pesquisa deve comparar a `query` com o `title` e o `artist` de cada faixa, sem distinção entre maiúsculas e minúsculas.
 - Chame `renderCatalogue` com o array filtrado para atualizar a lista.
 - Se a `query` estiver vazia, mostre o catálogo completo.

Desafios

- Implemente o comportamento do botão `btn-like`: ao clicar, altere o texto do botão para `"♥ Liked"` e adicione-lhe a classe CSS `liked`. Um segundo clique deve reverter ambas as alterações. Guarde o estado de like no próprio elemento com `dataset`.
- Adicione ao `index.html` botões de filtro por género (ex: "Todos", "Pop", "Jazz", "Hip-Hop"). Ao clicar em cada botão, filtre o catálogo pelo género correspondente e chame `renderCatalogue` com o resultado. Destaque o botão ativo com uma classe CSS `active`.